Low-Latency Deep Learning Inference for Order-Book Return Prediction

Stack: Python, PyTorch, ONNX, TensorRT, scikit-learn, LightGBM, Pandas, NumPy **Hardware:** NVIDIA A10 (24 GB), Lambda Labs

1. Overview

This project predicts short-horizon stock returns from order-book data, then deploys the trained neural network through ONNX and TensorRT and benchmarks inference latency and accuracy at several batch sizes and numeric precisions (FP32, FP16, INT8).

2. Data

- **Source:** Optiver *Trading at the Close* (Kaggle): 5,237,980 rows, 17 columns, 200 Nasdaq stocks over 481 trading days, one snapshot every 10 seconds during the final 10 minutes of each day.
- **Target:** the stock's 60-second-ahead WAP return relative to a synthetic index, in basis points (supervised regression). 88 rows with a missing target were dropped.
- **Split (chronological, whole trading days kept together to prevent leakage):**

Set	Days	Rows
Train	0–319	3,467,004
Validation (early stopping)	320–399	879,889
Test (reported results)	400–480	890,999

3. Feature engineering

19 features, including bid-ask spread, normalized book imbalance, WAP deviation from mid-price, auction imbalance relative to matched size (signed and unsigned), matched-to-book size ratio, gaps between auction prices (reference, near, far) and WAP, missing-value flags for near/far prices (only present from second 300), WAP returns over 10/30/60 seconds, and seconds in bucket. Features were median-imputed, standardized with train statistics, and clipped to ±10.

4. Predictive models

Model	Test MAE	Test RMSE	MAE vs. predict-zero
Predict-zero baseline	5.9936	8.9416	n/a
Ridge	5.9587	8.9039	-0.6%
Random Forest (400k-row sample)	5.9291	8.8678	-1.1%
PyTorch MLP (256-128-64, BatchNorm, dropout, L1 loss)	5.9019	8.8363	-1.5%
LightGBM (L1 objective)	5.8869	8.8056	-1.8%

Gains over the zero baseline are small, as expected for a noisy 60-second target. The MLP is competitive with the best tree model (within 0.25% MAE of LightGBM).

5. Deployment pipeline

1. Trained the MLP in PyTorch and exported it to **ONNX** with a dynamic batch dimension. ONNX Runtime outputs matched PyTorch (max absolute difference 3.8e-6).
2. Compiled **TensorRT** engines at three precisions: FP32, FP16 (built from a half-precision ONNX export), and INT8 (entropy calibration on rows disjoint from the evaluation rows; INT8-only build, FP16 disabled, to isolate its effect).
3. Verified each engine's accuracy on 100k holdout rows.

6. Benchmark methodology

- Input data is placed on the GPU once, so timings only cover model execution.
- 30 warm-up calls, then the mean of 200 calls (50 for the largest batch), timed with CUDA events and explicit synchronization (GPU launches are asynchronous, so wall-clock timing around a call would be misleading).
- Baselines: eager PyTorch, **PyTorch with a CUDA graph**, and a Random Forest on CPU.

7. Results

Accuracy (100k held-out test rows; a subset of the test period, so absolute MAE differs from Section 4):

Version	MAE	Change vs. PyTorch
PyTorch FP32	5.65745	n/a
TensorRT FP32	5.65745	0.0000%
TensorRT FP16	5.65744	-0.0002%
TensorRT INT8	5.66405	+0.12%

Latency (ms per call):

Batch	Eager PyTorch	PyTorch + CUDA graph	TRT FP32	TRT FP16	TRT INT8	RF (CPU)*
1	0.250	**0.025**	0.046	0.046	0.041	15.5
64	0.265	**0.042**	0.045	0.045	0.040	15.1
1,024	0.258	0.063	0.045	0.045	**0.040**	26.3
16,384	0.696	0.683	0.325	0.196	**0.071**	37.0
131,072	5.370	5.332	2.540	1.441	**0.505**	68.3

*Random Forest timed on CPU.

At 131,072 rows, TensorRT is 2.1× (FP32), 3.7× (FP16), and 10.6× (INT8) faster than eager PyTorch.

8. Key finding: a CUDA-graph baseline changes the interpretation

An initial comparison against eager PyTorch showed TensorRT roughly 5–6× faster at batch size 1. Eager PyTorch issues each small operation as a separate GPU command from Python, and for a model this small the fixed per-command overhead dominates the actual math. Adding a **CUDA graph** baseline (which records the command sequence once and replays it with a single launch) separated the two sources of speedup:

- **Small batches (1–64):** CUDA-graph PyTorch matches or beats TensorRT. The early speedup was launch-overhead removal, not faster computation.
- **Large batches (16k+):** CUDA graphs give no benefit, since overhead is negligible, while TensorRT still wins clearly. These gains come from layer fusion (fewer passes over GPU memory) and lower-precision arithmetic, which cut memory traffic.

Reporting against the strongest baseline is what makes the large-batch claim defensible.

9. Limitations

- We could eventually include optimizations and benchmarking for host-to-device data transfer and report tail latency.
- Single GPU (A10) and a single, small MLP architecture; no sequence or Transformer model was evaluated.
- TensorRT was not run with its own CUDA-graph support, so the batch-1 TensorRT vs. PyTorch comparison is not fully like-for-like.
- `torch.compile` was not evaluated (I was kinda tired at this point).
- Predictive gains over the baseline are modest; the project's contribution is the inference-optimization study, not the return prediction.